

GEO1000 – Python Programming – Assignment 1

Due: Monday, September 14, 2026 (18h00)

Introduction

You are expected to create 2 programs. All program files have to be handed in. Start with the files that are distributed via Brightspace. It is sufficient to modify the function definitions inside these files (replace *pass* with your own implementation).

Do not change the function signatures: i.e. keep their names, which & how many arguments the functions take, in which order the functions take arguments and what type they are supposed to return (as described).

This assignment is *preferably* made in groups of 2 (enroll with your group or individually in Brightspace), and your mark will count for your final grade of the course. Helping each other is fine. However, make sure that your implementation is your *own*.

This assignment in total can give you 100 points. Your assignment will be marked based on:

- whether your implementation does the correct things (as described in the assignment),
- whether your code is decent (e.g. use of proper variable names, indentation, etc), and
- whether your files are submitted as required (e.g. on time).

It is due: **Monday, September 14, 2026 (18h00).**

Note that if you submit your assignment after the deadline, some points will be removed. Up to 6 hours late: 5 points deducted from your final mark. From 6 up to 24 hours late: 10 points deducted from your final mark. For every additional day after that, another 30 points will be removed per day. An example: Assume you deliver the assignment at Monday, September 14, 2026, 18h15, the maximum amount of points that then can be obtained for the assignment is 95 ($100 - 5 = 95$).

Submit the resulting program files (`circles.py`, `triangle.py`) via Brightspace (your last submission will be taken into account, also for determining whether you are late). Upload **a zip file** that contains just the program files (with no folders/hierarchy inside)!

Make sure that each Python file handed in starts with the following comment (augment Authors and Studentnumbers with your own names and numbers):

```
# GEO1000 -- Assignment 1
# Authors:
# Studentnumbers:
```

1 Part 1: Circle-Circle Intersection Classifier (50 Points)

Write a program that determines the spatial relationship between two circles in a 2D Cartesian coordinate plane. A circle is defined by its center point (x, y) and radius $r \geq 0$. Given two circles

$C_1(x_1, y_1, r_1)$ and $C_2(x_2, y_2, r_2)$, classify their intersection state.

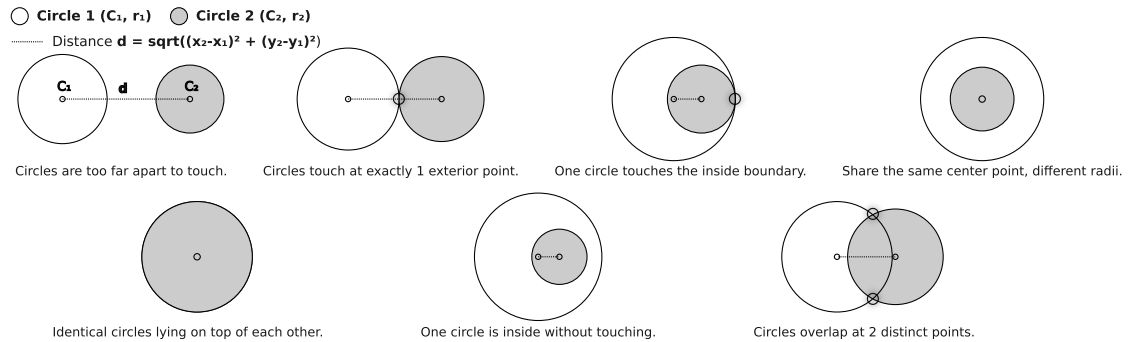


Figure 1: Circles and their spatial relationship

First, compute the Euclidean distance d between the center points of the two circles:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

The spatial relationship can then be determined by comparing d against the sum ($r_1 + r_2$) and the absolute difference ($|r_1 - r_2|$) of the two radii (using an $\text{eps} = 1\text{e-}7$ threshold, see Appendix on page 5 about the rationale for using this threshold):

1. **Identical / Coincident:** $d < \epsilon$ and $|r_1 - r_2| < \epsilon$ (infinite intersection points).
2. **Concentric & Disjoint:** $d < \epsilon$ and $|r_1 - r_2| \geq \epsilon$ (0 intersection points).
3. **Separate (Disjoint Outside):** $d > r_1 + r_2 + \epsilon$ (0 intersection points).
4. **Touch Externally:** $|d - (r_1 + r_2)| < \epsilon$ (1 intersection point).
5. **Intersect:** $|r_1 - r_2| < d < r_1 + r_2$ (2 intersection points).
6. **Touch Internally:** $|d - |r_1 - r_2|| < \epsilon$ (1 intersection point).
7. **Nested Inside:** $d < |r_1 - r_2| - \epsilon$ (0 intersection points).

Requirements & Functions

Function 1: `center_distance(x1, y1, x2, y2)`

- Must be a **fruitful function** that calculates and returns the Euclidean distance d as a float.

Function 2: `classify_circles(x1, y1, r1, x2, y2, r2)`

- Must be a **void function** (prints output message, returns None).
- **Modular Requirement:** You **must** call `center_distance` inside this function to calculate distance d .
- Uses `if-elif-else` conditional branching with $\text{eps} = 1\text{e-}7$ to classify the relationship and print a descriptive result message to `stdout`.

Restrictions

- You may **only** import `sqrt` from the `math` module (`from math import sqrt`).

Skeleton

Start from the following skeleton:

```
# GEO1000 -- Assignment 1 -- Circle Intersections
# Authors:
# Studentnumbers:

from math import sqrt

def center_distance(x1, y1, x2, y2):
    """Calculates and returns the Euclidean distance between two center points."""
    pass

def classify_circles(x1, y1, r1, x2, y2, r2):
    """Classifies and prints the spatial relationship between two circles."""
    pass

# Test execution calls
classify_circles(0.0, 0.0, 5.0, 10.0, 0.0, 3.0) # Separate
classify_circles(0.0, 0.0, 5.0, 8.0, 0.0, 3.0) # Touch externally
classify_circles(0.0, 0.0, 5.0, 3.0, 0.0, 4.0) # Intersect (2 points)
classify_circles(0.0, 0.0, 5.0, 2.0, 0.0, 3.0) # Touch internally
classify_circles(0.0, 0.0, 5.0, 1.0, 0.0, 2.0) # Nested inside
classify_circles(0.0, 0.0, 5.0, 0.0, 0.0, 5.0) # Identical
```

2 Part 2: Point-in-Triangle Test (50 points)

Write a program that tests whether a query point $P(x_p, y_p)$ lies **inside**, **outside**, or on the **boundary** of a triangle defined by three vertices $A(x_a, y_a)$, $B(x_b, y_b)$, and $C(x_c, y_c)$.

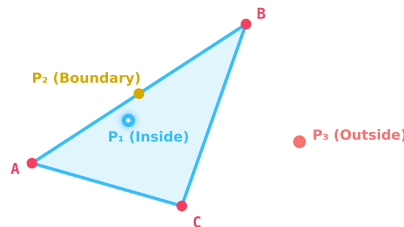


Figure 2: Triangle and 3 points (inside, boundary, outside)

The orientation of a point $P(x_p, y_p)$ relative to a directed line segment from $V_1(x_1, y_1)$ to $V_2(x_2, y_2)$ is determined using the 2D cross-product determinant D :

$$D(V_1, V_2, P) = (x_2 - x_1)(y_p - y_1) - (y_2 - y_1)(x_p - x_1)$$

- If $D > \epsilon$, P lies strictly to the **left** of the directed segment $V_1 \rightarrow V_2$.
- If $D < -\epsilon$, P lies strictly to the **right** of the directed segment $V_1 \rightarrow V_2$.
- If $|D| \leq \epsilon$, P is **collinear** with the segment $V_1 \rightarrow V_2$.

For triangle ABC , evaluate the cross products across all three directed edges:

$$D_{AB} = D(A, B, P), \quad D_{BC} = D(B, C, P), \quad D_{CA} = D(C, A, P)$$

Classification rules:

- **Inside:** All three cross products share the same non-zero sign (all $> \epsilon$ for CCW orientation, or all $< -\epsilon$ for CW orientation).
- **Boundary:** None of the cross products have opposite signs ($D_i > \epsilon$ and $D_j < -\epsilon$), and at least one cross product satisfies $|D_i| \leq \epsilon$.
- **Outside:** Cross products have mixed positive ($> \epsilon$) and negative ($< -\epsilon$) signs.

Requirements & Functions

When a tolerance value is needed, use an `eps = 1e-7` tolerance.

Function 1: `cross_product(x1, y1, x2, y2, px, py)`

- Must be a **fruitful function** returning the 2D cross product scalar D as a float.

Function 2: `triangle_area(xa, ya, xb, yb, xc, yc)`

- Must be a **fruitful function** calculating and returning the area of triangle ABC (as a float).
- **Modular requirement:** You **must** build on your `cross_product` helper function to compute the 2D cross product determinant between vertices, rather than re-implementing the determinant formula directly inside `triangle_area`.
- **Formula:** The area of the triangle is equal to half the absolute value of the 2D cross product determinant:

$$\text{Area} = \frac{1}{2} |\text{cross_product}(x_A, y_A, x_B, y_B, x_C, y_C)|$$

Function 3: `point_in_triangle(xa, ya, xb, yb, xc, yc, px, py)`

- Must be a **fruitful function** returning one of the exact lower-case string values: `inside`, `outside`, or `boundary`.
- **Modular requirement:** You **must** call `cross_product` to evaluate edge orientations.
- Must handle both counter-clockwise (CCW) and clockwise (CW) vertex orderings.

Tip for `point_in_triangle`: Instead of writing a web of nested `if/else` statements, evaluate your three cross products and set three simple boolean flags:

- `has_pos`: Did any edge give $D > \epsilon$?
- `has_neg`: Did any edge give $D < -\epsilon$?
- `has_zero`: Did any edge give $|D| \leq \epsilon$?

Then it is not difficult to get the required result:

- If you have *both* positive and negative signs (`has_pos` and `has_neg`), the point is `"outside"`.
- If it's not outside and you hit at least one edge (`has_zero`), it's on the `"boundary"`.
- Otherwise, it's safely `"inside"`.

Restrictions

- You may **only** import the standard `math` module (`import math`).

Skeleton

Start from the following skeleton:

```
# GEO1000 -- Assignment 1 -- Point-in-Triangle Test
# Authors:
# Studentnumbers:

import math

def cross_product(x1, y1, x2, y2, px, py):
    """Calculates 2D cross product determinant for orientation test."""
    pass

def triangle_area(xa, ya, xb, yb, xc, yc):
    """Calculates and returns the area of triangle ABC."""
    pass

def point_in_triangle(xa, ya, xb, yb, xc, yc, px, py):
    """Determines whether point P is inside, outside, or on the boundary of
    triangle ABC."""
    pass

# Test execution calls
print(point_in_triangle(0, 0, 10, 0, 0, 10, 2, 2))    # Expected output: inside
print(point_in_triangle(0, 0, 10, 0, 0, 10, 12, 5))   # Expected output: outside
print(point_in_triangle(0, 0, 10, 0, 0, 10, 5, 0))    # Expected output: boundary
print(point_in_triangle(0, 0, 10, 0, 0, 10, 0, 0))    # Expected output: boundary
```

Appendix: Floating-Point Numbers & Numerical Precision

Why floats are not mathematical real numbers

In pure mathematics, real numbers have infinite precision. However, computers store floating-point numbers (float in Python) using fixed-size binary memory structures (following the IEEE 754 standard).

A standard 64-bit float allocates memory into three parts:

- Sign bit (positive or negative)
- Exponent (scales the scale of the number)
- Mantissa / Significand (stores the significant precision digits, approx. 15–17 decimal digits)

Because floating-point numbers are represented in binary (base-2), many decimal fractions (base-10) and mathematical operations (like $\sqrt{x}$, division, or trigonometry) cannot be represented with infinite exactness. For instance, in Python:

```
0.1 + 0.2
# Output: 0.30000000000000004
```

Notice that $0.1 + 0.2$ is **not exactly equal** to 0.3 due to tiny rounding errors in the binary mantissa!

Why exact equality (==) fails in spatial geometry

When calculating distance d or cross product determinants D , tiny floating-point rounding errors accumulate. If you test exact equality:

```

if d == r1 + r2: # RISKY! May evaluate to False even when circles touch
    mathematically.
...
if cross_product == 0: # RISKY! May fail for collinear points due to 1e-16
    precision residual.
...

```

Checking `a == b` or `x == 0.0` directly with calculated float values will often cause false negatives and edge-case bugs in geometric algorithms.

How to compare floats: Using an Epsilon (ϵ) Tolerance

To compare floating-point numbers safely, in this assignment we will check if two values are ‘close enough’ within a tiny threshold called **epsilon** (ϵ). We define a small constant, for example `eps = 1e-7` ($10^{-7} = 0.0000001$).

1. Comparing if two floats a and b are equal ($a \approx b$):

Instead of `if a == b:`, check if the absolute difference is smaller than ϵ :

```

eps = 1e-7
if abs(a - b) < eps:
    # Treats a and b as equal

```

2. Comparing if a float x is zero ($x \approx 0$):

Instead of `if x == 0:`, check if its magnitude is smaller than ϵ :

```

eps = 1e-7
if abs(x) <= eps:
    # Treats x as 0 (e.g., collinear point or boundary condition)

```

3. Comparing inequalities safely ($x > 0$ vs $x < 0$):

To check if x is strictly positive or strictly negative beyond floating-point noise:

```

is_positive = x > eps
is_negative = x < -eps
is_zero     = abs(x) <= eps

```

Requirement for this assignment: You are expected to define and use an `eps = 1e-7` tolerance in both `circles.py` and `triangle.py` when testing boundary conditions, collinear points, or tangent circle cases.